

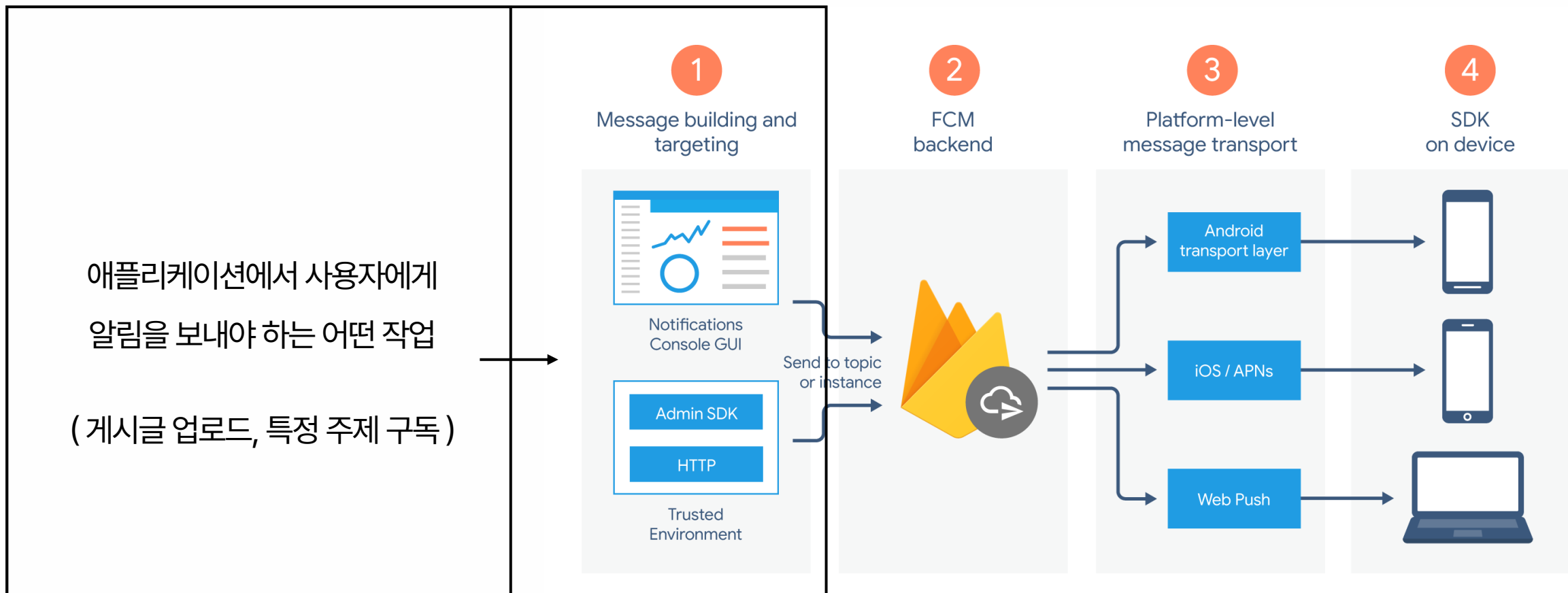
Observer Pattern

박연종

2026.05.20.

System Software Lab.

지난 시간에 살펴본 FCM 알림 발송 과정



두 작업을 분리하기 위해서는?

Observer Pattern

Publish / Subscribe

Pub/Sub Pattern

두 작업을 분리하기 위해서는?

Observer Pattern

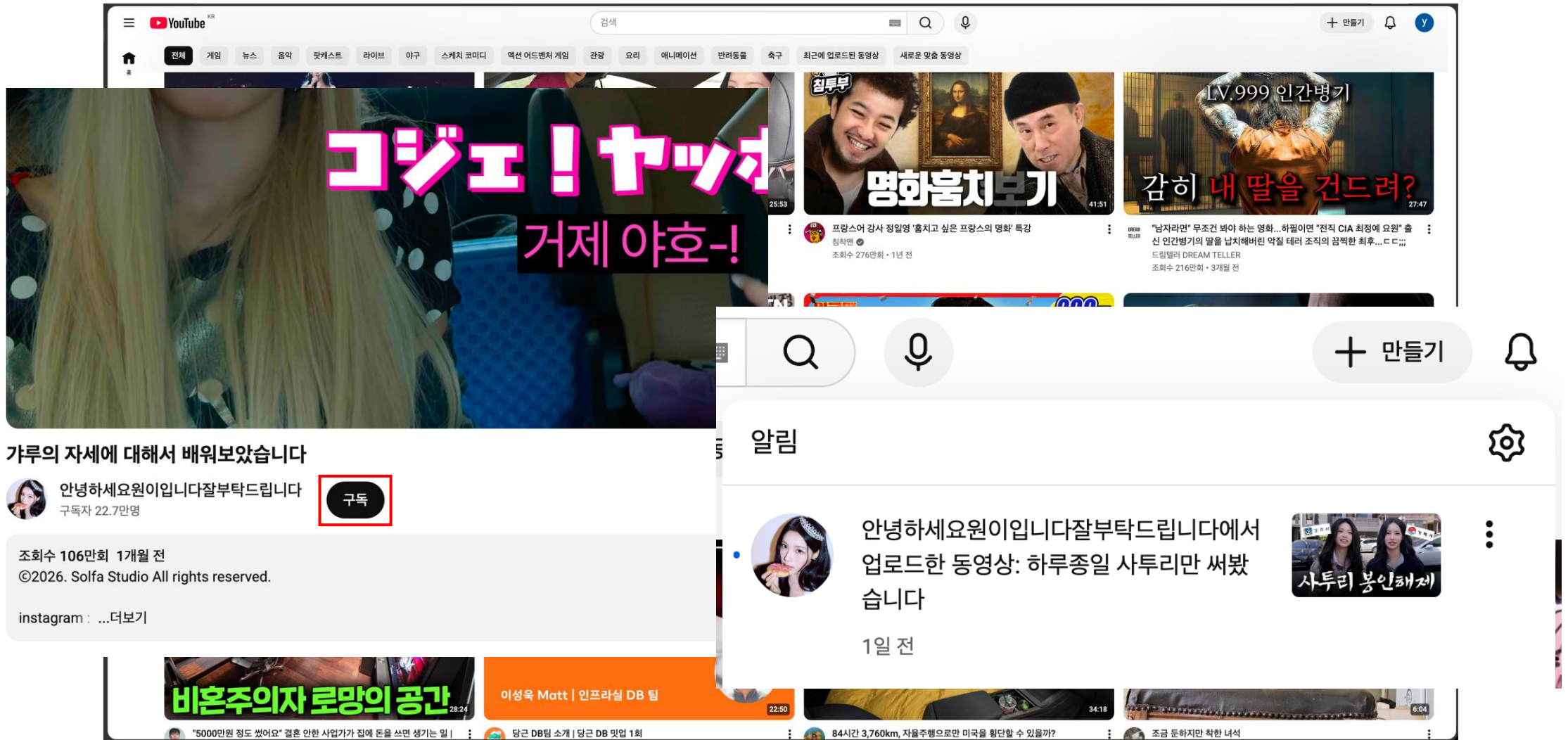
객체 상태가 변경되면 그 객체를 지켜보던
다른 객체들에게 자동으로 알림

Publish / Subscribe

Pub/Sub Pattern

발행자와 구독자가 서로를 모르는 상태로
중간자인 메시지 브로커를 통해 소식을 주고 받음

Observer Pattern

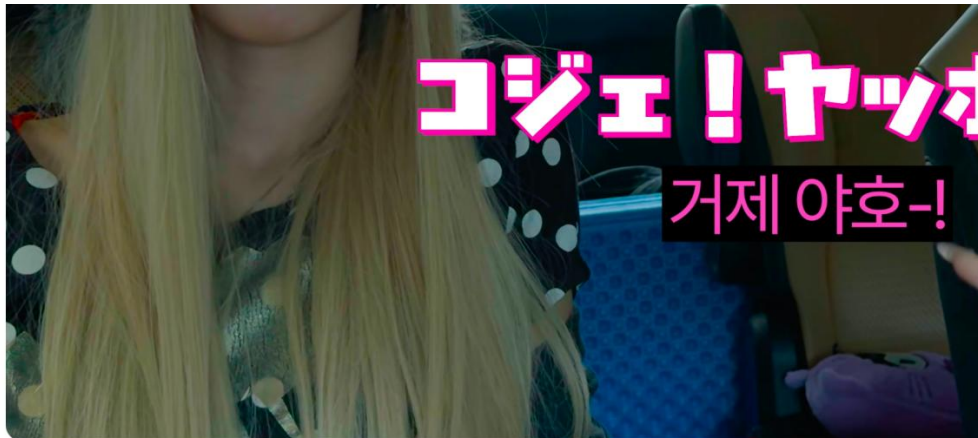


Observer Pattern

객체 상태가 변경되면 그 객체를 지켜보던 다른 객체들에게 자동으로 알리는

One-to-Many

일대다 의존성을 정의하는 디자인 패턴



가루의 자세에 대해서 배워보았습니다



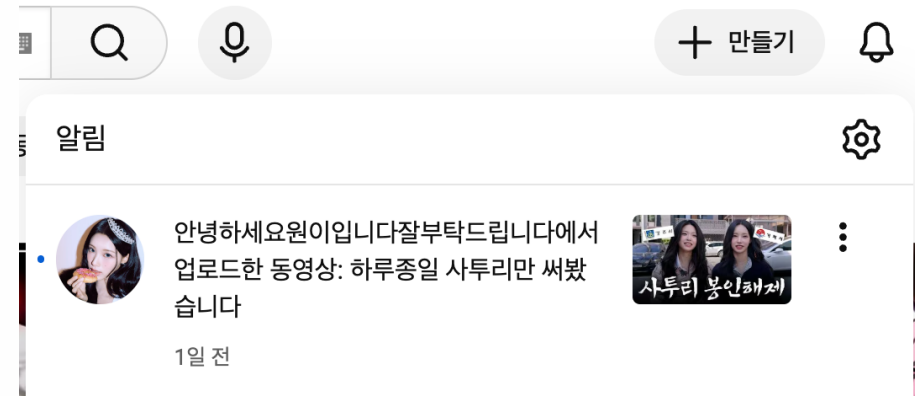
안녕하세요원이입니다
구독자 22.7만명

구독

조회수 106만회 1개월 전

©2026. Solfa Studio All rights reserved.

instagram : ...더보기



Observer Pattern

Subject 시간이 지나면 변경될 수 있는 중요한 상태를 가진 객체

Publisher Subject는 자신의 상태에 대한 변경을 다른 객체들에 알림을 보내는 역할도 맡음

Subscribers Publisher의 상태 변경에 대한 알림을 받고자 하는 객체



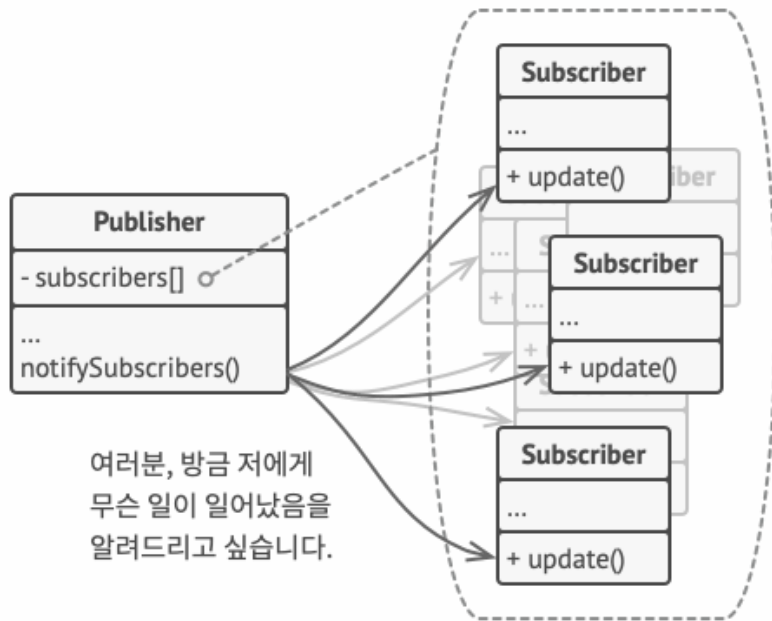
Observer Pattern은 개별 객체가 Publisher의 이벤트 알림을 구독 또는 구독 취소할 수 있는 매커니즘을 추가할 것을 제안

Observer Pattern

Subject 시간이 지나면 변경될 수 있는 중요한 상태를 가진 객체

Publisher Subject는 자신의 상태에 대한 변경을 다른 객체들에 알림을 보내는 역할도 맡음

Subscribers Publisher의 상태 변경에 대한 알림을 받고자 하는 객체



Publisher는 알림 메서드를 Subscriber로부터 호출해
각 객체에게 알림을 전송

Publisher는 모든 Subscriber를 호출하려면
Publisher와 Subscriber 간 결합도가 상승

이를 해결하기 위해서는 모든 Subscriber가
같은 인터페이스를 구현하고, Publisher가 인터페이스로만 통신

패턴 구조

Publisher 시간이 지나면 변경될 수 있는 중요한 상태를 가진 객체

다른 객체들에 관심 이벤트들을 발행

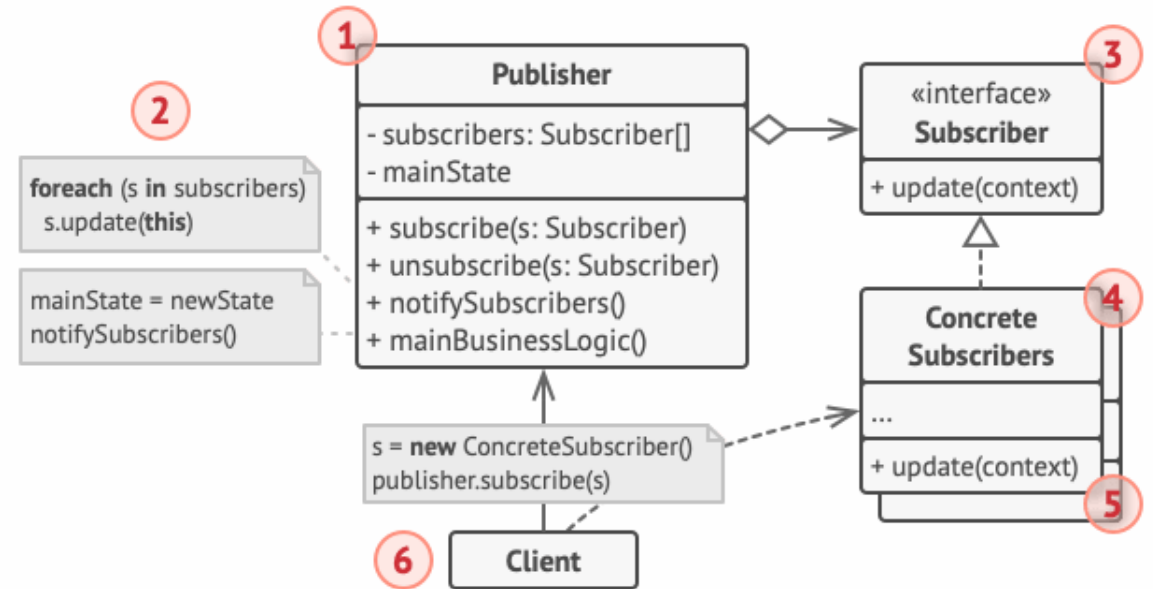
이벤트는 Publisher가 상태를 전환하거나 어떤 행동 실행 시 발생

Publisher 내에는 구독할 수 있는 매커니즘이 존재

구독 시 객체가 리스트에 추가되고, 구독 해제 시 리스트에서 삭제

새 이벤트 발생 시 Publisher가 리스트를 탐색한 후

Subscriber의 알림 메서드를 호출



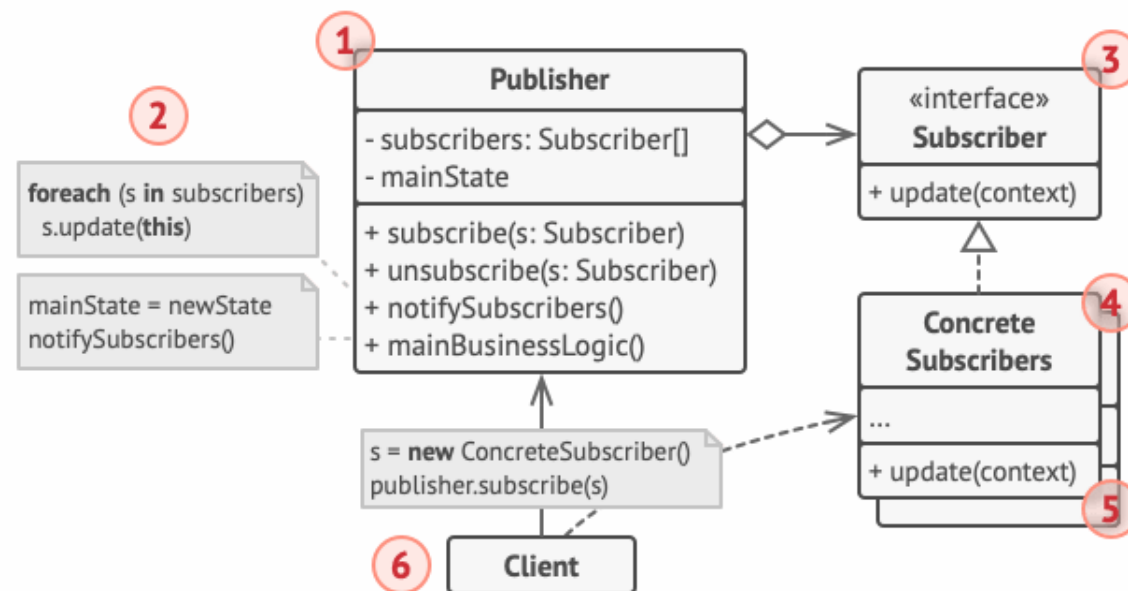
패턴 구조

Subscriber Publisher의 상태 변경에 대한 알림을 받고자 하는 객체

Subscriber 인터페이스는 대부분 단일 update 메서드로 구성
이벤트 세부 정보를 전달할 수 있도록 하는 여러 매개변수가 존재

Publisher는 Subscriber 인터페이스를 상속한 객체를 탐색하며
update 메서드를 호출

update 메서드 호출 시 Subscriber는 상태 갱신 등 여러 작업을 수행



느슨한 결합의 장점

Low coupling

Publisher와 Subscriber는 상호작용하고 있으나 서로의 구체적인 부분은 잘 모르는 상태

⇒ 느슨한 결합

Observer(Subscriber)를 언제든지 새로 추가하거나 제거할 수 있음

새로운 Observer라 하더라도 Subject(Publisher)를 변경할 필요가 없음

Subject(Publisher)와 Observer(Subscriber)는 서로 독립적으로 재사용할 수 있음

같은 Subject에 새로운 Observer 또는 같은 Observer에 다른 Subject를 붙여 사용할 수 있음

Subject(Publisher)나 Observer(Subscriber)가 변경되더라도 서로에게 영향을 미치지 않음

코드로 살펴보기

Publisher

```
void subscribe(Subscriber subscriber)  
void unsubscribe(Subscriber subscriber)  
void publish(Event event)
```

Subscriber

```
void update(Event event)
```

코드로 살펴보기

구체 클래스: SimplePublisher

```
public class SimplePublisher implements Publisher {
    private final List<Subscriber<? extends Event>> subscribers = new ArrayList<>();

    @Override
    public void subscribe(Subscriber<? extends Event> subscriber) {
        subscribers.add(subscriber);
    }

    @Override
    public void unsubscribe(Subscriber<? extends Event> subscriber) {
        subscribers.remove(subscriber);
    }

    @Override
    public void publish(Event event) {
        for (Subscriber<? extends Event> subscriber : subscribers) {
            if (subscriber.eventType().isAssignableFrom(event.getClass())) {
                notifySubscriber(subscriber, event);
            }
        }
    }

    private <E extends Event> void notifySubscriber(
        Subscriber<? extends Event> subscriber,
        Event event
    ) {
        Subscriber<E> typedSubscriber = (Subscriber<E>) subscriber;
        typedSubscriber.update((E) event);
    }
}
```

코드로 살펴보기

구체 클래스: NotificationSubscriber

```
public class NotificationSubscriber implements Subscriber<VideoUploadedEvent>
{
    private final ChannelSubscriptionRepository subscriptionRepository;
    private final NotificationScreen notificationScreen;

    public NotificationSubscriber(
        ChannelSubscriptionRepository subscriptionRepository,
        NotificationScreen notificationScreen
    ) {
        this.subscriptionRepository = subscriptionRepository;
        this.notificationScreen = notificationScreen;
    }

    @Override
    public void update(VideoUploadedEvent event) {
        List<User> users =
            subscriptionRepository.findUsersByChannelId(event.getChannelId());

        for (User user : users) {
            notificationScreen.show(user, event);
        }
    }
}
```

코드로 살펴보기

구체 클래스: VideoUploadService

```
public class VideoUploadService {  
    private final ChannelSubscriptionRepository subscriptionRepository;  
    private final Publisher publisher;  
  
    public VideoUploadService(  
        ChannelSubscriptionRepository subscriptionRepository,  
        Publisher publisher  
    ) {  
        this.subscriptionRepository = subscriptionRepository;  
        this.publisher = publisher;  
    }  
  
    public void uploadVideo(Channel channel, String videoTitle) {  
        VideoUploadedEvent event = new VideoUploadedEvent(  
            channel.getId(),  
            channel.getChannelName(),  
            videoTitle  
        );  
        publisher.publish(event);  
    }  
}
```

코드로 살펴보기

Client 클래스: YouTubeClient

```
void main() {
    publisher.subscribe(notificationSubscriber);

    Channel woni = new Channel(1L, "안녕하세요원이입니다잘부탁드립니다");
    Channel fs94 = new Channel(2L, "우리집정수기얼음나온다잉");

    User userA = new User(1L, "유정");
    User userB = new User(2L, "미나미");
    User userC = new User(3L, "제나");

    subscriptionRepository.subscribe(woni, userA);
    subscriptionRepository.subscribe(fs94, userB);
    subscriptionRepository.subscribe(woni, userC);

    Thread.sleep(2000L);
    videoUploadService.uploadVideo(woni, "미나미에게 위협을 느끼나요? | 10만 기념 Q&A");
    IO.println();

    Thread.sleep(2000L);
    videoUploadService.uploadVideo(fs94, "연애하고 싶은 모솔 남자들 필독 영상");
    IO.println();

    subscriptionRepository.unsubscribe(woni, userA);

    Thread.sleep(2000L);
    videoUploadService.uploadVideo(woni, "선배지만 친구가 하고 싶어");
}
```

```
/Users/patulus/.local/share/mise/installs/java/liberica-25/bin/java -javaagent:/Applications/IntelliJ IDEA
/Users/patulus/Documents/IdeaProjects/Seminar/demo/out/production/demo YouTubeClient
```

들어 주셔서 감사합니다

박연종	E-mail	park@duck.com
	LinkedIn	yeonjong-park
	Instagram	yeonjong.park
	GitHub	patulus